

EX.NO: 1

DATE:

AIM: To perform exploratory data analysis (EDA), data cleaning, RFM analysis, and customer classification using the CRISP-DM framework on an Online Retail dataset.

ALGORITHM / PROCEDURE:

1. **Data Understanding & Cleaning:** Load the online retail dataset, inspect data types, drop missing Description and CustomerID values, convert InvoiceDate to standard datetime format, and remove non-positive Quantity and UnitPrice values.
2. **Feature Engineering:** Compute TotalPrice by multiplying Quantity and UnitPrice. Calculate Recency, Frequency, and Monetary (RFM) values for each unique customer.
3. **Target Variable Definition:** Define a binary target HighValueCustomer corresponding to the top 25% of customers by monetary spend.
4. **Data Preparation:** Standardize feature values using StandardScaler and split the dataset into 80% training and 20% testing sets.
5. **Model Training & Evaluation:** Train a Random Forest Classifier on the processed features and evaluate performance using Accuracy, Precision, Recall, and Feature Importances.

PROGRAM:

```
import pandas as pd
import os

# load the dataset from the provided CSV file
retail_data_file_path = 'OnlineRetail.csv'
df_retail = pd.read_csv(retail_data_file_path, encoding = 'ISO-8859-1')

print("\n=====")
print("# Online Retail Customer Segmentation Dataset")
print("=====")
print(df_retail.head())
print(df_retail.info())
print(df_retail.isnull().sum())
```

Python

```
=====
# Online Retail Customer Segmentation Dataset
=====
  InvoiceNo StockCode      Description  Quantity \
0    536365    85123A  WHITE HANGING HEART T-LIGHT HOLDER      6
1    536365    71053      WHITE METAL LANTERN              6
2    536365    84406B  CREAM CUPID HEARTS COAT HANGER      8
3    536365    840296  KNITTED UNION FLAG HOT WATER BOTTLE      6
4    536365    84029E   RED WOOLLY HOTTIE WHITE HEART.      6

  InvoiceDate  UnitPrice  CustomerID  Country
0  12/1/2010 8:26      2.55    17850.0  United Kingdom
1  12/1/2010 8:26      3.39    17850.0  United Kingdom
2  12/1/2010 8:26      2.75    17850.0  United Kingdom
3  12/1/2010 8:26      3.39    17850.0  United Kingdom
4  12/1/2010 8:26      3.39    17850.0  United Kingdom
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
# 0  InvoiceNo    541909 non-null  object
# 1  StockCode   541909 non-null  object
# 2  Description  541909 non-null  object
# 3  Quantity    541909 non-null  int64
# 4  InvoiceDate  541909 non-null  object
# 5  UnitPrice   541909 non-null  float64
# 6  CustomerID  541909 non-null  float64
# 7  Country     541909 non-null  object
```

```
# 2. Convert InvoiceDate to datetime
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

print("DataFrame info after InvoiceDate conversion:")
print(df.info())
```

Python

```
DataFrame info after InvoiceDate conversion:
<class 'pandas.core.frame.DataFrame'>
Index: 406829 entries, 0 to 541908
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InvoiceNo        406829 non-null object
1   StockCode       406829 non-null object
2   Description     406829 non-null object
3   Quantity        406829 non-null int64
4   InvoiceDate     406829 non-null datetime64[ns]
5   UnitPrice       406829 non-null float64
6   CustomerID     406829 non-null int64
7   Country        406829 non-null object
dtypes: datetime64[ns](1), float64(1), int64(2), object(4)
memory usage: 27.9+ MB
None
```

```
# 3. Filter out invalid entries (Quantity <= 0 or UnitPrice < 0)
# Transactions with Quantity <= 0 are returns or adjustments
df = df[df['Quantity'] > 0]

# Transactions with UnitPrice < 0 are also typically returns or errors
df = df[df['UnitPrice'] >= 0]

print("DataFrame after filtering out invalid quantities and prices:")
print(f"New DataFrame shape: {df.shape}")
```

6]

Python

```
DataFrame after filtering out invalid quantities and prices:
New DataFrame shape: (397924, 8)
```

```
# 4. Calculate TotalPrice
df['TotalPrice'] = df['Quantity'] * df['UnitPrice']

print("DataFrame head with TotalPrice:")
display(df.head())

print("Final DataFrame info after all cleaning steps:")
print(df.info())
```

7]

Python

```
DataFrame head with TotalPrice:
```

```
Final DataFrame info after all cleaning steps:
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 397924 entries, 0 to 541908
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InvoiceNo        397924 non-null object
1   StockCode       397924 non-null object
2   Description     397924 non-null object
3   Quantity        397924 non-null int64
4   InvoiceDate     397924 non-null datetime64[ns]
```

```

# Calculate Recency, Frequency, Monetary (RFM) values

# 1. Determine the snapshot date (one day after the last transaction date)
snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)

# Group by CustomerID to calculate RFM metrics
rfm = df.groupby('CustomerID').agg(
    Recency=('InvoiceDate', lambda date: (snapshot_date - date.max()).days),
    Frequency=('InvoiceNo', 'nunique'),
    Monetary=('TotalPrice', 'sum')
).reset_index()

print("RFM DataFrame head:")
display(rfm.head())

print("RFM DataFrame info:")
print(rfm.info())

```

[13]

Python

... RFM DataFrame head:

... RFM DataFrame info:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4339 entries, 0 to 4338
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   CustomerID  4339 non-null  int64
1   Recency     4339 non-null  int64
2   Frequency   4339 non-null  int64
3   Monetary    4339 non-null  float64
dtypes: float64(1), int64(3)
memory usage: 135.7 KB
None

```

```

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Create a copy of the RFM data for classification
rfm_classification = rfm.copy()

# Define 'High-Value Customer' as top 25% of Monetary value
monetary_threshold = rfm_classification['Monetary'].quantile(0.75)
rfm_classification['HighValueCustomer'] = (rfm_classification['Monetary'] >= monetary_threshold)

print("RFM data with 'HighValueCustomer' target variable:")
display(rfm_classification.head())
print(rfm_classification['HighValueCustomer'].value_counts())

# Define features (X) and target (y)
X = rfm_classification[['Recency', 'Frequency', 'Monetary']]
y = rfm_classification['HighValueCustomer']

print("\nFeatures (X) head:")
display(X.head())
print("\nTarget (y) head:")
display(y.head())

```

[14]

Python

... RFM data with 'HighValueCustomer' target variable:

... HighValueCustomer

0	3254
1	1085

Name: count, dtype: int64

Features (X) head:

...

Target (y) head:

```

# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns, index=X.index)

print("Scaled Features (X) head:")
display(X_scaled_df.head())

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y # stratify to maintain
)

print(f"\nShape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

```

Python

Scaled Features (X) head:

```

Shape of X_train: (3471, 3)
Shape of X_test: (868, 3)
Shape of y_train: (3471,)
Shape of y_test: (868,)

```

```

# Model Training
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

print("RandomForestClassifier model trained successfully.")

```

Python

.. RandomForestClassifier model trained successfully.

```
[18] # Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"\nModel Accuracy: {accuracy:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Feature Importance
feature_importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)
print("\nFeature Importances:")
display(feature_importances)
```

...

Model Accuracy: 0.9988

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	651
1	1.00	1.00	1.00	217
accuracy			1.00	868
macro avg	1.00	1.00	1.00	868
weighted avg	1.00	1.00	1.00	868

Feature Importances:

RESULT: Thus the EDA and data cleaning using data science frameworks like CRISP-DM, SEMMA, KDD is implemented successfully.